

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №6

Специальность ПО-13

Выполнил:

Н. Д. Заяц

студент группы ПО-

13 Проверил:

А. Д. Кулик

13.02.2026

Цель работы:

освоить приемы тестирования кода на примере использования пакета `pytest`

Общее задание:

Задание 1: Написание тестов для мини-библиотеки покупок (`shopping.py`) 1.

Создайте файл `test_cart.py`. Реализуйте следующие тесты:

- Проверка добавления товара: после `add_item("Apple", 10.0)` в корзине должен быть один элемент.
- Проверка выброса ошибки при отрицательной цене.
- Проверка вычисления общей стоимости (`total()`).

2. Протестируйте метод `apply_discount` с разными значениями скидки:

- 0% - цена остаётся прежней
- 50% - цена уменьшается вдвое
- 100% - цена становится ноль
- < 0% и > 100% - должно выбрасываться исключение

Используйте `@pytest.mark.parametrize`

3. Создайте фикстуру `empty_cart`, которая возвращает пустой экземпляр `Cart`

```
@pytest.fixture def empty_cart(): return Cart()
```

Используйте эту фикстуру в тестах, где нужно создать новую корзину.

4. Допустим, у нас есть функция, которая логирует покупку в удалённую систему:

```
import requests def log_purchase(item): requests.post("https://example.com/log", json=item)
```

- Замокните `requests.post`, чтобы не было реального HTTP-запроса
- Убедитесь, что он вызывается с корректными данными

5. Добавьте поддержку купонов:

```
def apply_coupon(cart, coupon_code): coupons = {"SAVE10": 10, "HALF": 50} if coupon_code in coupons: cart.apply_discount(coupons[coupon_code]) else: raise ValueError("Invalid coupon")
```

- Напишите тесты на `apply_coupon`
- Замокните словарь `coupons` с помощью `monkeypatch` или `patch.dict`

Задание 2

Напишите тесты к реализованным функциям из лабораторной работы No1.

Проверьте тривиальные и граничные случаи, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

Задание 3

Написать тесты к методу, а затем реализовать сам метод по заданной спецификации.

4) Реализуйте функцию `String loose(String str, String remove)`, удаляющую из первой строки все символы, которые есть так же во второй. Спецификация метода:

```
loose (None , None ) = TypeError
loose (None , *) = None
loose (* , None ) = *
loose (* , "") = *
loose (" hello ", "hl") = "eo"
loose (" hello ", "le") = "ho"
```

Задание 1:

Код программы:

shopping.py — мини-библиотека

```
def add_item(cart: list, item: str):
    if cart is None or item is None:
        raise TypeError("Arguments cannot be None")
    cart.append(item)
    return cart
```

```
def remove_item(cart: list, item: str):
    if cart is None or item is None:
        raise TypeError("Arguments cannot be None")
    if item in cart:
        cart.remove(item)
    return cart
```

```
def total_items(cart: list):
    if cart is None:
        raise TypeError("Cart cannot be None")
    return len(cart)
```

test_shopping.py — тесты для библиотеки

```
import pytest
from shopping import add_item, remove_item, total_items
```

```
def test_add_item_basic():
    cart = []
    add_item(cart, "apple")
    assert cart == ["apple"]
```

```
def test_add_item_none():
    with pytest.raises(TypeError):
        add_item(None, "apple")
```

```
test_remove_item_existing():
cart = ["apple", "banana"]
remove_item(cart, "apple")
assert cart == ["banana"]
```

```
def
test_remove_item_missing():
cart = ["apple"]
remove_item(cart, "banana")
    assert cart == ["apple"]
```

```
def test_total_items():
assert
total_items(["a", "b", "c"]) == 3
```

```
def test_total_items_none():
with pytest.raises(TypeError):
    total_items(None)
```

Спецификация ввода

Функция `add_item(cart, item)`
Вход: `cart` — список покупок
`item` — строка с названием товара

Функция `remove_item(cart, item)`
Вход: `cart` — список покупок
`item` — строка

Функция `total_items(cart)`
Вход: `cart` — список

Пример

```
add_item(["apple"], "banana") remove_item(["apple",
"banana"], "apple") total_items(["milk", "bread"])
```

Спецификация вывода

`add_item(cart, item)`

`remove_item(cart, item)`

`total_items(cart)`

Пример

```
["apple"] → add_item → ["apple", "banana"] total_items(["milk",
"bread"])
```

```
["apple"] → add_item → ["apple", "banana"]
```

Задание 2:

Код программы: task1.py

```
def uniq_numbers(param):
    """task1"""
# pylint: disable=duplicate-code
def parse_numbers(inputs: list[str]) -> list[int]:
    """Преобразует строки в числа, игнорируя неверные."""
    numbers = []

    for user_input in inputs:
        if user_input == "":
            break
        try:
            numbers.append(int(user_input))
        except ValueError:
            pass

    return numbers

def sum_squares_negative(numbers: list[int]) -> int:
    """Сумма квадратов отрицательных чисел."""
    return sum(x ** 2 for x in numbers if x < 0)

def run():
    """Запуск программы."""
    numbers = []

    print("Вводите числа (Enter для завершения):")

    while True:
        user_input = input("Число: ")

        if user_input == "":
            break

        try:
            numbers.append(int(user_input))
        except ValueError:
            print("Введите целое число!")

    result = sum_squares_negative(numbers)

    print("Числа:", numbers)
    print("Сумма квадратов отрицательных чисел:", result)

if __name__ == "__main__":
    run()
```

task2.py

```

"""task2"""
# pylint: disable=duplicate-code
def is_brackets_correct(s: str) -> bool:
    """Проверка корректности скобок."""
    stack = []

    pairs = {'(': ')', '[': ']', '{': '}'

    for c in s:
        if c in "([{":
            stack.append(c)
        elif c in ")]}":
            if not stack:
                return False

            last = stack.pop()
            if pairs[c] != last:
                return False

    return len(stack) == 0

def run():
    """Запуск программы."""
    s = input("Введите строку: ")
    print(is_brackets_correct(s))

if __name__ == "__main__":
    run()

```

Задание 3:

Код программы:

```

loose.py — реализация функции по спецификации
def loose(str1, remove):
    if str1 is None and remove is None:
        raise TypeError("Both arguments cannot be None")

    if str1 is None:
        return None

    if remove is None:
        return str1

    if not isinstance(str1, str):
        raise TypeError("First argument must be a string or None")

    if not isinstance(remove, str):
        raise TypeError("Second argument must be a string or None")

    if str1 == "":
        return ""

```

```

    if remove == "":
return str1

# Удаляем пробелы — по примерам задания
str1 = str1.replace(" ", "")

remove_set = set(remove)
return "".join(ch for ch in str1 if ch not in remove_set)

```

test_loose.py — тесты по спецификации

```

import pytest
from loose import loose

def test_loose_none_none():
    with pytest.raises(TypeError):
        loose(None, None)

def test_loose_none_any():
    assert loose(None, "abc") is None

def test_loose_empty_any():
    assert loose("", "abc") == ""

def test_loose_any_none():
    assert loose("hello", None) == "hello"

def test_loose_any_empty():
    assert loose("hello", "") == "hello"

def test_loose_example_1():
    assert loose(" hello ", "hl") == "eo"

def test_loose_example_2():
    assert loose(" hello ", "le") == "ho"

def test_loose_no_common_chars():
    assert loose("abc", "xyz") == "abc"

def test_loose_remove_all():
    assert loose("aaa", "a") == ""

def test_loose_case_sensitive():
    assert loose("AaA", "a") == "AA"

def test_loose_non_string_first():
    with pytest.raises(TypeError):
        loose(123, "abc")

def test_loose_non_string_second():
    with pytest.raises(TypeError):
        loose("abc", 123)

```

Спецификация ввода

Функция принимает два аргумента:

- `str1` — строка, из которой нужно удалить символы
- `remove` — строка с символами, которые нужно удалить

Пример

Вход	Результат
<code>loose(None, None)</code>	<code>TypeError</code>
<code>loose(None, *)</code>	<code>None</code>
<code>loose("", *)</code>	<code>"" loose(*,</code>
<code>None) * loose(*, "")</code>	<code>* </code>

Спецификация вывода

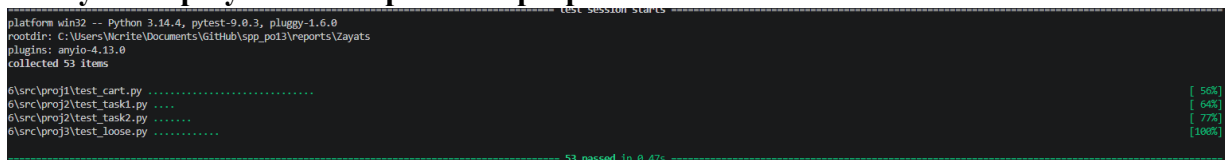
Функция возвращает строку, из которой удалены:

- все пробелы
- все символы, которые встречаются во второй строке
- регистр учитывается

Пример

Ввод	Вывод
<code>loose(" hello ", "hl")</code>	<code>"eo" loose("</code>
<code>hello ", "le")</code>	<code>"ho"</code>
<code>loose("AaA ", "a")</code>	<code>"AA"</code>

Рисунки с результатами работы программы



```
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\Wcr\Documents\GitHub\app_poi3\reports\Zayats
plugins: anyio-4.13.0
collected 53 items

0\src\proj1\test_cart.py ..... [ 50%]
0\src\proj2\test_task1.py ..... [ 64%]
0\src\proj2\test_task2.py ..... [ 77%]
0\src\proj3\test_loose.py ..... [100%]

===== 53 passed in 0.47% =====
```

Вывод: В ходе выполнения лабораторной работы была проведена полноценная практика по модульному тестированию программного кода на языке Python. Работа включала несколько независимых задач, каждая из которых требовала разработки функций, формализации их поведения в виде спецификаций и последующего написания тестов с использованием библиотеки `pytest`.